

Pav Bhaji Metadata Classification – Data Analysis Report

Objective

Classify whether the image attached to an Instagram post is Pav Bhaji (`1`) or not (`0`) **from the post JSON metadata only**. Image bytes are never read by the model. The image folders are used only once to attach the supplied ground-truth labels to JSON records.

Data audit

`pavbhaji.json` contains 1,500 Instagram metadata objects. Labels could be joined for 452 of them by matching the filename in `display\_url` with the filenames under `images/0` and `images/1`.

Item	Result
Labelled records used	452
Pav Bhaji (`1`)	183 (40.5%)
Not Pav Bhaji (`0`)	269 (59.5%)
Unlabelled JSON records excluded from fitting	1,048
Records with an empty caption	0
Duplicate post IDs	0
Exact duplicate model-text records	11

The classes are moderately imbalanced, so Pav Bhaji precision, recall, and F1 are reported in addition to accuracy. A majority-class-only baseline would achieve 59.5% accuracy.

Every sampled post was collected around `#pavbhaji`: 100% of class `1` and 99.3% of class `0` contain a Pav Bhaji spelling variant in the caption/tags. That feature is consequently a weak standalone discriminator and is deliberately not treated as proof of the image label.

Candidate features and relevance

Feature	Processing	Why it can help
Caption text	Lowercase normalization; word TF-IDF 1–2 grams	Dish names, ingredients, restaurants, and contextual phrases can distinguish a post actually about Pav Bhaji from a generic hashtagged post.
Hashtags/tags	Field-prefixed tokens in the same TF-IDF text	Food-specific and location/restaurant tags provide concise semantic signals. Prefixing keeps tag vocabulary distinguishable from caption words.
Character n-grams	TF-IDF character 3–5 grams	Robust to `pav-bhaji`, `pav_bhaji`, spelling variants, emojis, and compound hashtags.
Caption length and tag count	Standardized numeric features	Posting style differs between detailed food reviews and broad tag-spam; useful as a small supporting signal.
Likes and comments	`log1p` then standardization	Engagement may proxy account/post style, but it is weak and can change over time, so it should not dominate a production model.
`is_video`, width/height ratio	Standardized numeric features	Lightweight post-format context; not image content.
Pav Bhaji mention flag	Normalized binary feature	Handles explicit spelling variants, but is low-value here because the collection method already selected for that term.

Excluded fields: raw image pixels/URLs, post ID, shortcode, owner ID, timestamp, and location. IDs can cause memorization; image URLs/pixels violate the text-only requirement; timestamps and locations need a stronger product rationale before use.

Method

1. Match labels from `images/{0,1}/<filename>` to `display\_url` filenames.
2. Extract the first caption, tags, engagement fields, dimensions, and video flag from JSON.
3. Normalize text, preserving special `hashtag\_` and `tag\_` tokens.
4. Make a deterministic stratified 75/25 train/test split (`random\_state=42`).
5. Fit a class-balanced logistic regression on word TF-IDF, character TF-IDF, and scaled numeric metadata.

Logistic regression is a good small-data baseline: sparse text is handled efficiently, probabilities are available for thresholding, and coefficients can be inspected. `outputs/important\_features.csv` records the most influential fitted features.

Held-out evaluation

The following is from one untouched, stratified test set of 113 posts; it is not training-set performance.

Metric	Value
Accuracy	0.6372
Pav Bhaji precision	0.5362
Pav Bhaji recall	0.8043
Pav Bhaji F1	0.6435
ROC-AUC	0.6463

Confusion matrix (rows are actual classes; columns are predicted classes):

Actual \ Predicted	Not Pav Bhaji	Pav Bhaji
Not Pav Bhaji	35	32
Pav Bhaji	9	37

The model improves slightly over the 59.5% majority baseline and has high Pav Bhaji recall, but precision remains modest. This is expected because both classes were preselected using `#pavbhaji`, leaving unusually similar metadata. For a workflow where false positives are costly, raise the probability threshold above 0.50 after validation; where review is available, the current threshold is appropriate for high-recall candidate retrieval.

Reproducibility and deliverables

Run from the project directory:

```
```bash
cd machinelearningchallenge
python3 -m pip install -r requirements.txt
python3 main.py --dataset dataset --output outputs
```

The command creates:

- `outputs/metrics.json` – metrics and full classification report
- `outputs/heldout\_predictions.csv` – image filename, label, predicted class, and Pav Bhaji probability for every held-out post
- `outputs/important\_features.csv` – interpretable model coefficients

For production, collect labels from posts not prefiltered by `#pavbhaji`, use a time- or account-grouped validation split to limit leakage, and tune the classif

ication threshold on a validation set aligned to the desired false-positive/false-negative tradeoff.